

Matrices Dispersas

Una implementación en Python

Definición

En análisis numérico y computación científica, una matriz dispersa (o matriz rala) es una matriz en la que la mayoría de los elementos son cero. No hay una definición estricta sobre la proporción de elementos con valor cero para que una matriz se considere dispersa, pero un criterio común es que el número de elementos no nulos sea aproximadamente igual al número de filas o columnas.

$$\begin{bmatrix} 0 & 9 & 0 & 0 & 0 & 4 & 0 & 0 \\ 0 & 0 & 6 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 5 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 3 & 0 \\ 0 & 0 & 6 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Una matriz se almacena típicamente como un array bidimensional. Cada entrada en el array representa un elemento a_{ij} de la matriz y se accede a ella mediante los dos índices i y j . Convencionalmente, i es el índice de fila, numerado de arriba a abajo, y j es el índice de columna, numerado de izquierda a derecha. Para una matriz de mn , la cantidad de memoria requerida para almacenar la matriz en este formato es proporcional a mn (sin tener en cuenta el hecho de que también se deben almacenar las dimensiones de la matriz).

En el caso de una matriz dispersa, podemos ahorrar una cantidad sustancial de memoria si solo almacenamos las entradas no nulas. Dependiendo del número y la distribución de las entradas no nulas, podemos usar diferentes estructuras de datos y obtener grandes ahorros de memoria en comparación con el enfoque básico.

Si nos centramos en cómo construimos matrices como COO (lista de coordenadas). COO almacena una lista de tuplas $\langle \text{valor}, \text{fila}, \text{columna} \rangle$ de cada valor no nulo.

Por ejemplo, dada la siguiente matriz dispersa de 5x5 con 4 valores no nulos:

```
1 m = [[0, 0, 1, 0, 0],
2       [0, 0, 0, 0, 0],
3       [-1, 0, 0, 0, 0],
4       [1, 2, 0, 0, 0],
5       [0, 0, 0, 0, 0]]
```

Podemos ver que:

- El elemento 1 está en la fila 0, columna 2
- El elemento -1 está en la fila 2, columna 0
- El elemento 1 está en la fila 3, columna 0
- El elemento 2 está en la fila 3, columna 1

Por lo tanto, su representación COO es:

```
1 d = [[1, -1, 1, 2],
2       [0, 2, 3, 3],
3       [2, 0, 0, 1]]
```

Ventajas del formato COO

- Facilita la conversión rápida entre formatos dispersos
- Permite entradas duplicadas (ver ejemplo)
- Conversión muy rápida hacia y desde aquellos formatos que soportan modificaciones eficientes

Desventajas del formato COO

- No soporta directamente:
 - Operaciones aritméticas
 - Slicing (trozeado)

Ejercicios

Te pedimos que escribas las siguientes funciones para poder trabajar con matrices dispersas en una función principal (lo explicaremos al final de este documento).

Función `introducir_matriz()`-> `list`

Implementa una función `introducir_matriz()`-> `list`. Esta función pedirá al usuario la matriz desde la consola. El usuario escribirá cada línea de la matriz en una sola línea en la consola. Si el usuario introduce una línea vacía, la función dejará de pedir más líneas. Tendrás que comprobar si la matriz está correctamente formada, es decir, si todas las filas tienen el mismo número de columnas. La matriz no tiene que ser una matriz cuadrada.

Función `leer_matriz()`-> `list`

Implementa una función `leer_matriz()`-> `list`. Esta función leerá la matriz desde un fichero. Para empezar, el usuario tendrá que escribir la ruta del fichero y la función tendrá que leer el fichero y procesarlo en una matriz. Supondremos que cada fichero contiene una sola matriz. De la misma manera que el ejercicio anterior, comprueba si cada fila de la matriz contiene la misma cantidad de columnas.

Función `crear_coo()`-> `list`

Implementa la función `crear_coo()`-> `list`. Esta función no tiene parámetros y devuelve una matriz COO vacía, es decir, devuelve una lista con 3 listas vacías.

Función `normal_a_coo(matriz: list)`-> `list`

Implementa la función `normal_a_coo(matriz: list)`-> `list`. Esta función toma un parámetro, una matriz de representación estándar de $m \times n$ y devuelve la misma matriz en su formato COO $\langle valor, fila, columna \rangle$, es decir, devuelve una lista de $3 \times q$, donde q es el número de elementos no nulos.

Ejemplo, para la siguiente entrada:

```
1 m = [[0, 0, 1, 0, 0],
2       [0, 0, 0, 0, 0],
3       [-1, 0, 0, 0, 0],
4       [1, 2, 0, 0, 0],
5       [0, 0, 0, 0, 0]]
```

La salida debería ser:

```
1 coo = [[1, -1, 1, 2],
2         [0, 2, 3, 3],
3         [2, 0, 0, 1]]
```

Función `coo_a_normal(coo: list, filas: int, columnas: int)-> list`

Implementa la función `coo_a_normal(coo: list, filas: int, columnas: int)-> list`. Esta función toma tres parámetros: el primero es una matriz dispersa, el segundo es el número de filas y el tercero corresponde al número de columnas. Devuelve una representación estándar de la matriz de *fxc*.

Ejemplo, para la siguiente entrada:

```
1 coo = [[1, -1, 1, 2],
2         [0, 2, 3, 3],
3         [2, 0, 0, 1]]
```

La salida debería ser:

```
1 m = [[0, 0, 1, 0, 0],
2       [0, 0, 0, 0, 0],
3       [-1, 0, 0, 0, 0],
4       [1, 2, 0, 0, 0],
5       [0, 0, 0, 0, 0]]
```

Función `existe_en(coo: list, fila: int, columna: int)-> int`

Implementa la función `existe_en(coo: list, fila: int, columna: int)-> int`. Esta función también toma tres parámetros: el primero corresponde a una matriz dispersa en formato COO, el segundo es el número de fila y el tercero es el número de columnas. Si la matriz dispersa tiene un valor no nulo en la posición `coo[fila,columna]`, entonces la función devolverá el índice de columna de este elemento; de lo contrario, devolverá `-1`.

Ejemplo, para la siguiente entrada:

```
1 coo = [[1, -1, 1, 2],
2         [0, 2, 3, 3],
3         [2, 0, 0, 1]]
4 fila = 2
5 columna = 0
```

La salida debería ser:

```
1 1
```

Y, para la siguiente entrada:

```
1 coo = [[1, -1, 1, 2],
2         [0, 2, 3, 3],
3         [2, 0, 0, 1]]
4 fila = 3
5 columna = 4
```

La salida debería ser:

```
1 -1
```

Función

`insertar_o_incrementar(coo: list, fila: int, columna: int, numero: float)-> None`

Implementa la función `insertar_o_incrementar(coo: list, fila: int, columna: int, numero: float)-> None`. Esta función tomará cuatro parámetros: La matriz dispersa en formato COO, un índice de fila, un índice de columna y un número. La función insertará o incrementará el valor contenido en `coo_fila,columna` dependiendo de las siguientes condiciones:

- Si $coo_{fila,columna} \neq 0 \rightarrow$ añadiremos el número a $coo_{fila,columna}$
- Si $coo_{fila,columna} = 0 \rightarrow$ asignaremos $coo_{fila,columna}$ al número

Ejemplo, para la siguiente entrada:

```
1 coo = [[1, -1, 1, 2],
2         [0, 2, 3, 3],
3         [2, 0, 0, 1]]
4 fila = 2
5 columna = 0
6 numero = 5
```

La salida debería ser:

```
1 coo = [[1, 4, 1, 2],
2         [0, 2, 3, 3],
3         [2, 0, 0, 1]]
```

Y, para la siguiente entrada:

```
1 coo = [[1, -1, 1, 2],
2         [0, 2, 3, 3],
3         [2, 0, 0, 1]]
4 fila = 3
5 columna = 3
6 numero = -5
```

La salida debería ser:

```
1 coo = [[1, -1, 1, 2, -5],
2         [0, 2, 3, 3, 3],
3         [2, 0, 0, 1, 3]]
```

Función `suma_dispersas(coo1: list, coo2: list)-> list`

Implementa la función `suma_dispersas(coo1: list, coo2: list)-> list`. Esta función tomará dos matrices dispersas como parámetros, ambas en formato COO. La función calcula y devuelve una nueva matriz, también en formato COO, que corresponde a la suma de ambas matrices de entrada.

Consejo: Para que esta función funcione, debes iterar sobre todos los elementos de `coo1` e insertarlos en la nueva matriz. Luego, harás lo mismo con los elementos de `coo2`.

Ejemplo, para la siguiente entrada:

```
1 coo1 = [[1, -1, 1, 2],
2         [0, 2, 3, 3],
3         [2, 0, 0, 1]]
4 coo2 = [[1, 1, 2],
5         [0, 1, 3],
6         [2, 1, 1]]
```

La salida debería ser:

```
1 coo = [[1, -1, 1, 2, -5],
2        [0, 2, 3, 3, 3],
3        [2, 0, 0, 1, 3]]
```

Función `imprimir_matriz(matriz: list)-> None`

Implementa la función `imprimir_matriz(matriz: list)-> None`. Esta función toma un solo parámetro, una representación estándar de una matriz de $m \times n$ y la imprime en la pantalla.

Programa principal

Dadas dos matrices, $m1$ y $m2$, debes transformarlas en formato COO. Luego, debes crear una tercera matriz, en formato COO, que contenga la suma de las dos matrices anteriores (es decir, $m1 + m2$). Finalmente, muestra la matriz resultante en su representación normal. Una de las matrices deberá leerse desde un fichero y la otra deberá introducirse desde la consola.

Ejemplo, para la siguiente entrada:

```
1 m1 = [[0, 1, 0, 0],
2        [0, 0, 0, 0],
3        [0, 0, 2, 0],
4        [0, 0, 0, 0]]
5 m2 = [[0, 0, 0, 0],
6        [0, 5, 0, 0],
7        [0, 0, 0, 0],
8        [0, 0, 0, 0]]
```

La salida debería ser:

```
1 La suma de m1 y m2 es:
2 0 1 0 0
3 0 5 0 0
4 0 0 2 0
5 0 0 0 0
```